Soft Spatial Querying on JSON Data Sets

Paolo Fosci(✉) and Giuseppe Psaila(✉)

University of Bergamo, Viale Marconi 5, 24044 Dalmine, (BG), Italy
{paolo.fosci,giuseppe.psaila}@unibg.it
http://www.unibg.it

Abstract. *JSON* (JavaScript Object Notation) has become popular for exchanging data sets over the Internet. Many data sets are "geo-tagged", since they represent spatial entities. As an effect, spatial analysts have to perform spatial queries on *JSON* data sets. While working with large data sets, crisp (on/off) spatial relations could be marginally effective; instead, soft relations and "soft spatial querying" could be the right tools, because they reveal the extent of a given spatial relation. In this paper, we present the recent evolution of *J-CO-QL*$^+$, the query language of the *J-CO* Framework (under development at University of Bergamo, Italy) towards soft spatial querying on geo-tagged *JSON* data sets.

Keywords: Geo-tagged JSON documents · Soft spatial querying · Fuzzy operators and fuzzy spatial relations

1 Introduction

The advent of Open Data portals has pushed forward the distribution of geographical information: public authorities publish authoritative data sets that describe territories. In this context, *JSON* (JavaScript Object Notation) has become very popular to represent data sets over the Internet. In our opinion, this popularity is due to its simple syntax, which makes *JSON* data sets easy to process. Consequently, often analysts and data engineers have to work with *JSON* data sets, possibly describing "geo-tagged" data, i.e., data that are associated with geometrical descriptions of real-world entities on the Earth surface.

In this regard, *GeoJSON* is playing an important role, since it is a standard format that relies on *JSON* as hosting syntax to represent "geographical information layers". Many data sets concerning spatial data are provided as *GeoJSON* documents, on which spatial analysts have to perform "spatial queries".

When spatial querying is performed on large data sets, often spatial relations between spatial entities cannot be on/off (or crisp), because what it matters is the "extent" to which the relation is met. Furthermore, the query itself could be explained in a vague or tolerant way, so as to catch unexpected situations. Therefore, we are moving towards "soft spatial querying".

At University of Bergamo (Italy), we are devising the *J-CO* Framework [19]: its goal is to provide analysts with tools to flexibly acquire, integrate and query

S. Chiusano et al. (Eds.): ADBIS 2022, LNCS 13389, pp. 223–237, 2022.
https://doi.org/10.1007/978-3-031-15740-0_17

JSON data sets, in a way that is independent of the platform that provides and stores data sets. In the original design of the *J-CO-QL* query language [3], we were focused on the native support for spatial querying on geo-tagged *JSON* data sets. Later, we realized that soft querying could be a valuable tool for analysts: we started extending *J-CO-QL* towards evaluating membership degrees of *JSON* documents to fuzzy sets in order to perform soft querying (see [11]). We understood that, to fully introduce soft-querying capabilities, it was necessary to revise the whole language, that has become *J-CO-QL*$^+$: constructs have been unified, re-arranged and made more intuitive and flexible, for better supporting soft querying.

The contribution of this paper is to present the recent constructs to perform soft spatial querying, through the unified `JOIN` statement (until [11,19] there were two distinct `JOIN` statements). The new `JOIN` statement, which pairs documents coming from two data sets, now supports fuzzy sets; furthermore, soft spatial relations can be now evaluated, thus enabling soft spatial querying.

The paper is organized as follows. Section 2 introduces the relevant background for the paper. Section 3 provides the main contribution of the paper. Finally, Sect. 4 draws the conclusions and highlights possible future works.

2 Background

In this section, we briefly present the background of our work, i.e., basic notions on fuzzy sets (Sect. 2.1), the literature on soft querying databases (Sect. 2.2) and a short introduction to the *J-CO* Framework (Sect. 2.3).

2.1 Brief Introduction to Fuzzy Sets

The notion of Fuzzy Set (and related theory) was introduced by Zadeh in [20]; here, we report a brief introduction.

Let us denote by U a non-empty universe, either finite or infinite.

Definition 1. *A fuzzy set $A \subseteq U$ is a mapping $A : U \rightarrow [0, 1]$. The value $A(x)$ is referred to as the* membership degree *of the item $x \in U$ to the A fuzzy set.*

We can say that a fuzzy set A in U is characterized by a membership function $A(x)$ that associates each item $x \in U$ with a real number in the range $[0, 1]$; this value denotes the degree with which x is a member of A, also called "membership degree". Specifically, if $A(x) = 0$, this means that x does not belong at all to A; if $0 < A(x) < 1$, this means that x belongs to A only partially, with an extent that is the membership degree; $A(x) = 1$ means that x fully belongs to A.

As a example, consider the universe U of scientific papers. The fuzzy set called *RelevantPapers* denotes those papers that are relevant for the research conducted by a researcher named Jane: she judges the relevance of a paper x, by expressing its membership degree; clearly, $RelevantPapers(x) = 1$ means that the x paper is fully relevant; given two papers x_1 and x_2, if $RelevantPapers(x_1) = 0.8$ and $RelevantPapers(x_1) > RelevantPapers(x_2)$,

this means that x_1 is more relevant than x_2, even though x_1 is not fully relevant to Jane's research (its membership degree is 0.8).

Some properties can be easily introduced.

A fuzzy set is empty if and only if its membership function is identically zero for each $x \in U$.

Two fuzzy sets A and B are equal, denoted as $A = B$, if and only if $A(x) = B(x)$ for all $x \in U$.

The classical set-oriented operations can be extended to fuzzy sets.

Definition 2. *Consider two fuzzy sets A and B in U.*

Considering the items $x \in U$ in the intersection $I = A \cap B$, they have the membership function $I(x) = min(A(x), B(x))$.

Considering the items $x \in U$ in the union $S = A \cup B$, they have the membership function $S(x) = max(A(x), B(x))$.

Considering the items $x \in U$ in the complement of A, i.e., $C_A = U - A$, they have the membership function $C_A(x) = 1 - A(x)$.

Soft conditions linguistically express the fact that an item belongs to fuzzy sets. Consequently, the `AND` operator is mapped to the fuzzy intersection, the `OR` operator is mapped to the fuzzy union, the `NOT` operator is mapped to the fuzzy complement.

For example, considering again Jane's research, we could think about a second fuzzy set called $HighlyCitedPapers$, which denotes the level of citations; so, Jane could look for papers x such that

$$HighlyCitedPapers(x) \texttt{ AND } RelevantPapers(x),$$

which is an example of soft condition. The resulting membership degree denotes the relevance of a paper x w.r.t. the condition; so it can be used to rank results.

In our work, we consider the universe U of *JSON* documents. Thus, given a document $d \in U$, the membership degree $A(d)$ denotes the extent with which d belongs to the A fuzzy set.

2.2 Related Work

Soft querying on data sets is not a novel topic: in the past, it was deeply investigated on top of relational databases. The first approach was to work on an already existing database; under this hypothesis, the query language should support soft querying, but the underlying database model cannot change; these query languages extend the concept of "selection condition", as argued by [2], so as to formulate "soft queries" in which selection conditions can rely on vague predicates; the goal is to obtain queries that are tolerant to thresholds, which is a typical problem in classical crisp querying; for example, given a non-soft predicate `income >=100000` to select people whose annual income is no less than 100000 Euros, a person with 99999 Euros as annual income would not be selected. By adopting fuzzy sets, it is possible to express the concept of `WantedIncome` in

a soft way, so that 99999 Euros is considered not fully wanted, i.e., has a membership degree slightly less than 1. Since relational databases were considered, many extensions of SQL towards soft querying were proposed: they preserve the underlying classical relational data model but provide capabilities of soft querying table rows through fuzzy-set theory. Since it is impossible to be exhaustive here, we mention *SQLf* (see [8,9]) and the attempt to implement it (in [13]); the second proposal we mention is *FQUERY for Access* (see [15] and [21]), which was designed to work within Microsoft Access; finally, we mention the proposal called *SoftSQL* (see [4–6]), which also covers the definition of user-defined "linguistic predicates" through a dedicated statement, to be used in soft selection conditions in the `SELECT` statement.

Removing the hypothesis of working on top of an existing relational database, the data model can be extended towards "fuzzy-relational databases" (see [7] and [17]) for representing uncertain and imprecise data directly within the database, by means of "fuzzy values". Here, we mention *FSQL*, presented in [12,14], which is the most remarkable proposal, in our opinion.

The recent advent of the notion of *JSON* document stores (NoSQL databases specifically designed to store *JSON* documents) seems stimulating novel research on soft querying, this time on *JSON* data sets and stores.

The work [1] proposed *fMQL*, an extension of *MQL* (the *MongoDB* Query Language). The authors worked under the hypothesis that *JSON* documents are previously labeled with "fuzzy labels".

Recently, [16] proposed an extension of the *MongoDB* data model towards a fuzzy *JSON* document store, supporting fuzzy values in single fields.

2.3 The *J-CO* Framework

The *J-CO* Framework [3,18,19] is a tool able to retrieve, integrate and query collections of *JSON* documents either stored in *JSON* document stores or directly provided by Web sources. The core of the framework is *J-CO-QL*$^+$, a novel declarative query language specifically designed to query heterogeneous *JSON* data sets, by natively dealing with spatial relations between geo-tagged documents. The *J-CO* Framework is composed of several software tools; the interested reader can refer to [19] for a complete description. Here, we just mention the *J-CO-QL*$^+$ *Engine*, which actually executes *J-CO-QL*$^+$ queries/scripts; in particular, it is worth noticing that this is independent of any *JSON* document store, while it is able to retrieve data from and store data to several stores (but it processes data independently of specific processing capabilities provided by specific *JSON* stores).

Here, we briefly introduce the data and execution models.

Data Model. *J-CO-QL*$^+$ works on collections of standard *JSON* documents. Two special root-level fields, named `~fuzzysets` and `~geometry`, play a specific role. The `~fuzzysets` field works as a map $fsn \rightarrow md$, where fsn is a fuzzy-set name and md is the corresponding membership degree (see Fig. 3c); this way, the degrees of membership to multiple fuzzy sets of a document can be

simultaneously represented. The `~geometry` field represents spatial geometries [3,19], by relying on the `GeometryCollection` format of *GeoJSON*.

Execution Model. We define the concept of *query-process state* as a tuple $s = \langle tc, IR, DBS, FO \rangle$. Its members have the following meaning: $s.tc$ is called *temporary collection* of documents; $s.IR$ (*Intermediate Results*) is a local database where the process can temporarily store collections; $s.DBS$ is a set of database descriptors, used to connect to *JSON*databases; $s.FO$ is the set of *fuzzy operators* (see Sect. 3.2) defined throughout the query. All members in the initial state s_0 are empty sets.

A query (or script) is a sequence of instructions. An instruction i_j works on the $s_{(j-1)}$ query-process state and generates a new query-process state s_j, where one member can be changed. In particular, consider the temporary collection tc: i_j receives the $s_{(j-1)}.tc$ temporary collection as input and may generate a new $s_j.tc$ temporary collection as output.

3 Case Study and *J-CO-QL*$^+$ Script

We now present the main contribution of the paper. Section 3.1 introduces the case study. Then, Sect. 3.2 presents how *J-CO-QL*$^+$ deals with fuzzy concepts. Section 3.3 shows how to pre-process *GeoJSON* documents for soft querying. Finally, Sect. 3.4 shows how to perform soft spatial queries in *J-CO-QL*$^+$.

3.1 Case Study

Suppose that an analyst has to face the following problem.

Problem 1. *Consider a European Union (EU) region R and its provinces $P(R)$, whose geographical description is provided. Consider also a registry of Gross Domestic Product ($GDP(p)$) about $P(R)$, in which GDP in 2017 and 2018 are reported for a EU province p. Finally, consider a* GeoJSON *document that describes the set L of lakes that are either partially or totally in the territory of R. We want to look for provinces $p \in P(R)$ such that there is a lake $l \in L$ partially falling in p's territory (but in a significant way), such that p had a significant increase in GDP from 2017 to 2018.*

The goal of the analyst is to discover if there is a relation between the fact a significant part of a lake falls into the territory of a given province p and the province increase in GDP.

In the rest of the paper, we refer to the following territory and data sets:

- R is the Italian region named Lombardia, where our University is located.
- EU NUTS describe administrative regions in the EU. They were downloaded from the Eurostat website[1] as a shape file. The data set encompasses many

[1] https://ec.europa.eu/eurostat/web/gisco/geodata/reference-data/administrative-units-statistical-units/nuts.

administrative levels: Level 0 corresponds to countries; Level 1 corresponds to macro-regions; Level 2 corresponds to regions; finally, Level 3 corresponds to provinces. The shape file was converted to geo-tagged *JSON* documents where the geometry is compliant with *GeoJSON* through QGIS, from the 2016 EU NUTS, we filtered the 12 Level-3 ones (provinces) in Lombardy.
- Registry data about provinces were downloaded as an Excel file from the Eurostat *data browser*[2]. Only rows concerning the 12 provinces in Lombardy were selected and converted into *JSON* documents.
- Data about lakes were downloaded from Regione Lombardia Open Data portal[3]. They were provided as shape files and then converted to *GeoJSON* through QGIS. The *GeoJSON* document encompasses 5469 lakes.

Listing 1 *J-CO-QL*$^+$ script, part 1.

```
1. CREATE FUZZY OPERATOR IncreasingGDP
     PARAMETERS   gdp0 TYPE Float, gdp1 TYPE Float
     PRECONDITION gdp0 > 0 AND gdp1 > 0
     EVALUATE     100*(gdp1-gdp0) / gdp0
     POLYLINE     [ (0, 0.0), (1, 0.8), (3, 1.0)];

2. CREATE FUZZY OPERATOR InterestingLake
     PARAMETERS   ownership TYPE Float
     PRECONDITION ownership IN_RANGE [0, 1]
     EVALUATE     ownership
     POLYLINE     [ (0, 0), (0.25, 0), (0.3, 1), (0.8, 1), (0.85, 0), (1, 0) ];

3. USE DB AbdisDb ON SERVER MongoDB 'http://127.0.0.1:27017';

4. GET COLLECTION GeojsonLakes@AbdisDb;

5. EXPAND
     UNPACK WITH ARRAY .features
     ARRAY .features TO .LakeGeoData;

6. FILTER
     CASE WHERE WITH .LakeGeoData.item
       GENERATE
         SETTING GEOMETRY .LakeGeoData.item.geometry
         BUILD {
           .properties: .LakeGeoData.item.properties
         };

7. SAVE AS Lakes;
```

In Problem 1, two concepts are naturally imprecise. The first one is "Increasing GDP": when a GDP is increasing in a significant way? Small variations of GDP are absolutely normal, both in a positive and in a negative way; so, it is necessary to define this imprecise concept in a possibly tolerant way.

The second imprecise concept is a "significant portion" of lake in the territory of a province p. When is a lake portion significant, in such a way it is not marginal and the lake is not completely contained in the territory of the province p?

The *J-CO-QL*$^+$ script to address Problem 1 is presented in Listings 1 and 2. We will present it in the remainder of this section.

[2] https://ec.europa.eu/eurostat/databrowser/explore/all/all_themes?lang=en.
[3] https://www.dati.lombardia.it/Territorio/Lago-10000-CT10/qm9t-uzst.

Listing 2 *J-CO-QL*$^+$ script, part 2.

```
8. JOIN OF COLLECTIONS GeoNuts@AbdisDb AS G, NutsInfo@AbdisDb AS I
     SET GEOMETRY LEFT
     CASE WHERE  WITH   .G.properties.NUTS_ID,    .G.properties.CNTR_CODE,
                        .G.properties.LEVL_CODE,  .I.NUTSCode
          AND .G.properties.NUTS_ID    = .I.NUTSCode
          AND .G.properties.CNTR_CODE  = "IT"
          AND .G.properties.LEVL_CODE  = 3
       GENERATE
         CHECK FOR FUZZY SET ImprovingWealth
           USING IncreasingGDP(TO_FLOAT(.I.GDP2017), TO_FLOAT(.I.GDP2018))
         BUILD {
           .nutsName: .G.properties.NUTS_NAME,
           .nutsId: .G.properties.NUTS_ID,
           .gdp2017: TO_FLOAT(.I.GDP2017),
           .gdp2018: TO_FLOAT(.I.GDP2018)   };

9. JOIN OF COLLECTIONS temporary AS N, Lakes AS L
     ON GEOMETRY INTERSECT
     SET GEOMETRY INTERSECTION
     SET FUZZY SETS LEFT ALL, HOW_INCLUDE (RIGHT) AS OwnedLake
     CASE WHERE KNOWN FUZZY SETS ImprovingWealth, OwnedLake
           AND WITH .L.properties
       GENERATE
         CHECK FOR
           FUZZY SET WishedLakes
             USING InterestingLake (MEMBERSHIP_OF (OwnedLake)),
           FUZZY SET Wanted
             USING ImprovingWealth AND WishedLakes
         ALPHACUT 0.7 ON Wanted
         BUILD {
           .nutsName  : .N.nutsName,
           .nutsId    : .N.nutsId,
           .gdp2017   : .N.gdp2017,
           .gdp2018   : .N.gdp2018,
           .lake      : .L.properties.NOME_LG,
           .rank      : MEMBERSHIP_OF (Wanted)    }
         DEFUZZIFY;

10.  SAVE AS RankedNuts@AbdisDb;
```

3.2 Definition of Fuzzy Concepts

The first two instructions in Listing 1 define two "fuzzy operators". In *J-CO-QL*$^+$, a fuzzy operator is an operator that evaluates the membership degree of a *JSON* document to a fuzzy property or relation. Remember that we have to define two concepts, i.e., "increasing GDP" and "interesting lake"; the two fuzzy operators defined on lines 1 and 2 correspond to these two concepts.

Increasing GDP. The instruction on line 1 of Listing 1 defines the fuzzy operator called `IncreasingGDP`. Its goal is to evaluate if a province had a relevant increase of GDP in two consecutive years. Details are presented hereafter.

- The `PARAMETERS` clause specifies the formal parameters of the operator. Specifically, two floating-point parameters are defined, called `gdp0` and `gdp1`.
- The `PRECONDITION` clause specifies a condition that must be met by actual parameters, before evaluating the operator (otherwise an exception is raised and the evaluation is stopped). Specifically, GDPs must be greater than zero.

- The `EVALUATE` clause specifies a mathematical expression evaluated on the parameters: the value it returns is used as x-axis value of the membership function specified in the `POLYLINE` clause. Specifically, the expression computes the percentage of variation of the two GDPs.
- The `POLYLINE` clause specifies the membership function used to obtain the membership degree. It is defined as a sequence of points, where x-coordinates are free, while y-coordinates are in the range $[0, 1]$. Between two consecutive points, the function is a segment connecting the two points. For x values less than the leftmost point, its y-coordinate is returned as membership degree; similarly, for x values greater than the rightmost point.
  Figure 1a depicts the polyline. Notice that the membership degree increases from 0 to 0.8 up to 1% of GDP increase, becoming significantly interesting after 1% (degree greater than 0.8). From 1% to 3%, it slowly reaches 1: any further increase is considered equally of interest.

InterestingLake. The instruction on line 2 defines the second fuzzy operator, named `InterestingLake`. Its goal is to evaluate if a lake is interesting (for the analysis) on the basis of the ownership degree of its area by the province.

The operator receives one single formal parameter: it is called `ownership`, since it is the degree with which the province owns the area of a lake. The expression in the `EVALUATE` clause does not make any computation on the value of the parameter, which is used as it is as x-coordinate on the membership function.

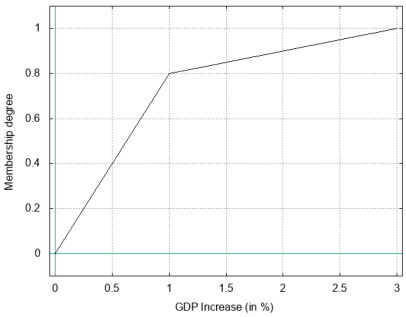
The `POLYLINE` clause defines the membership function, as depicted in Figure 1b. The trapezoidal shape excludes those lakes whose area marginally falls into a province, because the membership degree is 0 up to an ownership degree of 25%. It is 1 in the range $[0.3, 0.8]$. After 0.85 it is 0 again (this way, we exclude those lakes that are substantially fully contained in the territory of a province).

Notice that by means of the two fuzzy operators, we are able to quantify the relevance of GDP increase in our context, as well as the relevance of a lake (based on the percentage of its area owned by a province). Nevertheless, intermediate membership degrees allow for being tolerant and consider borderline situations with respect to thresholds (0.3 and 0.8, for the `InterestingLake` operator).

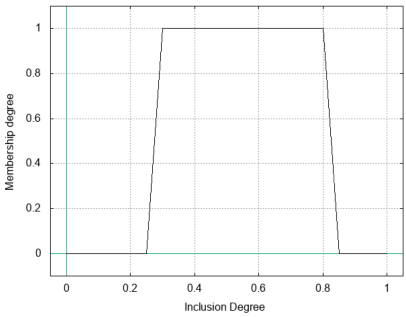
3.3 Processing a GeoJSON Document

In Listing 1, lines from 3 to 6 acquire and process a *GeoJSON* document that describes lakes, stored in a *MongoDB* database called `AdbisDb`. This part of the script is not strictly connected with soft querying: hereafter, we provide a short introduction (the interested reader can refer to our previous works [10,19]).

- The `USE DB` instruction on line 3 connects to the *MongoDB* server and to the desired database.
- The `GET COLLECTION` instruction on line 4 retrieves the content of the collection named `GeojsonLakes`, stored in the `AdbisDb` database; the collection



(a) Membership function of the `IncreasingGDP` fuzzy operator.



(b) Membership function of the `InterestingLake` fuzzy operator.

Fig. 1. Membership functions of the fuzzy operators.

contains one single *GeoJSON* document describing lakes; an excerpt of this document is reported in Figure 2a: notice that it is a single document, the root-level field of which, called `features`, is an array of documents; each nested document describes a "feature", i.e., a spatial entity with "properties" (the `properties` field) and "geometry" (the `geometry` field).

- The `EXPAND` instruction on line 5 unnests documents in the `features` array: the instruction generates a new collection of *JSON* documents, which contains as many documents as the ones contained in the `features` array. In details, each unnested document contains all fields in the source document, except the `features` array field; in its place, the `LakeGeoData` field is added, which contains two fields named `item` and `position`, where the former one contains the item actually unnested from within the original array, while the latter denotes the position occupied by the unnested item in the array.
- The `FILTER` instruction on line 6 restructures the documents in the temporary collection generated by line 5. Specifically, the `SETTING GEOMETRY` clause takes the `geometry` field nested within the `LakeGeoData` field as the geo-tagging of the document (as an effect, the `~geometry` field is added to the document). Then, the `BUILD` block restructures the document, putting the nested `properties` field at the top level. An example of documents contained in the new temporary collection is reported in Figure 2b.
- The instruction on line 7 saves the temporary collection into the *IR* (Intermediate Results) database, so as to reuse it later during the query process.

3.4 Soft Spatial Querying

The second part of the *J-CO-QL*$^+$ script (reported in Listing 2) actually performs a soft query on the data. Provided that line 10 actually saves the final temporary collection into the `AdbisDb` database, the key instructions are on lines 8 and 9.

(a) Excerpt of the *GeoJSON* document in the `GeojsonLakes` collection.

(b) Example of document in collection Lakes after line 6.

Fig. 2. Sample documents concerning lakes.

Processing NUTS. The first step is to process data about EU NUTS for the 12 considered provinces, stored within the `GeoNuts` collection in the `AdbisDb` database. Figure 3a reports a sample document: they follow the same structure of *GeoJSON* features; nevertheless, they were previously pre-processed, so they already respect the *J-CO-QL*$^+$ data model (see the `~geometry` field).

The `NUTSInfo` collection (stored within the same database) actually contains registry data, about provinces. A sample document is reported in Figure 3b: notice that this is a very simple *JSON* document, which provides extra data to those provided by documents in the `GeoNUTS` collection (see Figure 3a).

The `JOIN` instruction on line 8 of Listing 2 pairs documents in the two collections, so as to extend a document g in the `GeoNUTS` collection (aliased as `G`) with the corresponding document i in the `NutsInfo` collection (aliased as `I`). The instruction behaves as follows.

- For each pair (g, i), a new document d is generated, where the `G` field contains the source g document, while the `I` field contains the source i document.
- The `SET GEOMETRY` clause denotes (if present) the geometry to assign to the d document. In this case, the left geometry is taken, so as to keep geometries of NUTS. As an effect, now d has the `~geometry` field too.
- The `CASE WHERE` clause selects those documents the analyst is interested in. Specifically, documents describing an Italian and level-3 (provinces) NUTS, that has been paired to the corresponding registry data.
- The `CHECK FOR FUZZY SET` clause evaluates the membership degree of the d document to the `ImprovingWealth` fuzzy set: this is done by means of the `USING` sub-clause, where a "soft condition" is expressed. Specifically, the

`IncreasingGDP` fuzzy operator is used (see Sect. 3.2). As a consequence, the d document is further extended with the `~fuzzysets` field.

– Finally, the `BUILD` block simplifies the document, as shown in Figure 3c (notice that GDP values are converted from strings, as in the source `NutsInfo` collection, to numerical values through the `TO_FLOAT` built-in function).


```
{
  "properties" : {
    "NUTS_ID"    : "ITC47",
    "LEVL_CODE"  : 3,
    "CNTR_CODE"  : "IT",
    "NAME_LATN"  : "Brescia",
    "NUTS_NAME"  : "Brescia",
    "MOUNT_TYPE" : 2,
    "URBN_TYPE"  : 2,
    "COAST_TYPE" : 3,
    "FID"        : "ITC47"
  },
  "~geometry"  : {
    "type"       : "Polygon",
    "coordinates": [
      [ [ 10.515752, 46.343224  ],
        …,               [ …, … ] ] ]
  }
}
```


(a) Example of document in the `GeoNUTS` collection.


```
{
    "NUTSCode"    : "ITC47",
    "NUTSLabel"   : "Brescia",
    "Area"        : 4786,
    "Pop2017"     : 1262.5,
    "Pop2018"     : 1265.9,
    "GDP2017"     : "42151.46",
    "GDP2018"     : "43311.54"
}
```


(b) Example of document in the `NUTSInfo` collection.


```
{
  "nutsId"     : "ITC47",
  "nutsName"   : "Brescia",
  "gdp2017"    : 42151.46,
  "gdp2018"    : 43311.54,
  "~fuzzysets" : {
    "ImprovingWealth": 0.975217040222694
  },
  "~geometry"  : {
    "type"       : "Polygon",
    "coordinates": [
      [ [ 10.515752, 46.343224  ],
        …,               [ …, … ] ] ]
  }
}
```


(c) Example of document after line 8.


```
{
  "nutsId"     : "ITC47",
  "nutsName"   : "Brescia",
  "gdp2017"    : 42151.46,
  "gdp2018"    : 43311.54,
  "lake"       : "Garda (Lago di)",
  "rank"       : 0.975217040222694,
  "~geometry"  : {
    "type"       : "Polygon",
    "coordinates": [
      [ [ 10.515752, 46.343224  ],
        …,               [ …, … ] ] ]
  }
}
```


(d) Example of document after line 9.

Fig. 3. Sample documents concerning NUTS and obtained by Listing 2

Cross-Analyzing Lakes and Provinces. We are now ready to cross-analyze lakes and provinces, by performing a soft spatial query (on line 9 of Listing 2).

– The instruction builds pairs (n, l), where n comes from the temporary collection generated by line 8 (see the `temporary` keyword), while l comes from the `Lake` collections generated by line 6 and temporarily stored within the IR database; the collections are aliased as `N` and `L`, respectively.
– The `ON GEOMETRY` clause expresses a "spatial condition" to generate pairs. Specifically, a pair (n, l) is selected if n's and l's geometries intersect.

– A new document d is generated, having the `N` field (containing the n document) and the `L` field (containing the l document).
  Based on the `SET GEOMETRY` clause, the `~geometry` field is added too, containing the intersection of the two source geometries (i.e., the portion of lake that falls into the territory of the province).
– To complete the novel d document, it is necessary to deal with fuzzy sets. First of all, fuzzy sets whose membership was already evaluated for input documents must be considered (recall, looking at Figure 3c, that documents generated by line 8 had the membership degree to the fuzzy set called `IncreasingWealth`). Second, only here it is possible to exploit "fuzzy spatial relations", i.e., gradual relations concerning geometries of source documents. Both these tasks are done by the `SET FUZZY SETS` clause:
  - `LEFT ALL` specifies that all membership degrees to fuzzy sets evaluated for documents coming from the left collection (in this case, those generated by line 7) must be kept (i.e., the `IncreasingWealth` fuzzy set);
  - the membership degree to the `OwnedLake` fuzzy set is obtained by means of the `HOW_INCLUDE` function, which provides the degree of inclusion of the intersection in the area of the right (as specified within parentheses) geometry. As a result, d is further extended with the `~fuzzysets` field, which contains two fields (i.e., `IncreasingWealth` and `OwnedLake`).
  - Alternatively, specific fuzzy-set names and the side they come from (i.e., `LEFT` or `RIGHT`) could be specified.

J-CO-QL$^+$ provides three functions that evaluate a fuzzy spatial relation between two source geometries:

- `HOW_INCLUDE(RIGHT)` (resp. `HOW_INCLUDE(LEFT)`) provides the degree the intersection is included in the right (respectively left) geometry;
- `HOW_MEET(RIGHT)` (resp. `HOW_MEET(LEFT)`) provides the degree the right perimeter touches the left one (resp., the left perimeter touches the right one);
- `HOW_INTERSECT()` provides the degree the area of the intersection is contained in the area of the united geometries.

We can now continue explaining the remainder of line 9.

– The `CASE WHERE` condition selects those documents so far obtained for which the membership degrees to the `ImprovingWealth` and `OwnedLake` fuzzy sets has been evaluated (see the `KNOWN FUZZY SETS` predicate).
– Within the `GENERATE` block, two `FUZZY SET` branches are specified in the `CHECK FOR` clause.
  Specifically, the first one evaluate a fuzzy set that is called `WishedLakes`: its membership degree denotes the degree of interest the lake has for the query. This is done by means of the `InterestingLake` fuzzy operator (see Sect. 3.2), which receives the membership degree to the `OwnedLake` fuzzy set, obtained through the `MEMBERSHIP_OF` function (remember that a lake is interesting only if the percentage of its area that falls into the territory of the province is approximately between 30% and 80%).

- On the basis of the last evaluated fuzzy set, the second `FUZZY SET` branch evaluates the membership degree to the last `Wanted` fuzzy set. The `USING` soft condition says that analyst is interested in those documents that describe a province that shows `IncreasingWealth` and whose territory contains `WishedLakes`. Notice the use of the `AND` logical operator, now in the fuzzy version (that returns the minimum membership degree, see Definition 2).
- The `ALPHACUT` clause filters documents on the basis of their membership degree to the `Wanted` fuzzy set: specifically, documents whose membership degree is no less than 0.7 (i.e., high satisfaction of the condition) are kept.
- The `BUILD` block generates the final structure of documents, flattening them. Furthermore, the `rank` field is added, taking the membership degree to the `Wanted` fuzzy set as value, because the document is "de-fuzzified" (i.e., the `~fuzzysets` field is removed). Figure 3d reports a sample output document.

The reader can notice how the last `USING` clause expresses the soft condition in a linguistic way, if fuzzy set names are properly chosen: indeed, we are looking for provinces that have shown increasing wealth and contain a wished lake. The final membership degree is used to rank retrieved documents, so as to express the relevance of the document to the query. Even if the soft condition is not directly expressed on a fuzzy set directly obtained by means of a spatial fuzzy relation, nevertheless, the `WishedLakes` fuzzy set derives from the `OwnedLake` fuzzy set, evaluated in the `SET FUZZY SETS` clause when source documents are actually paired. So, this is a simple yet complete example of soft spatial querying performed on *JSON* documents through *J-CO-QL*$^+$.

To conclude this section, we highlight the novelty introduced in the presented version of the language. First of all, the general organization of sub-clauses associated to the `WHERE` selection condition has been significantly improved. Furthermore, in place of having two distinct `JOIN` statements, one for spatial join and one for non-spatial join, now a unified `JOIN` statement is provided; not only, apart from the fact that this statement now supports fuzzy-set evaluation, novel fuzzy spatial functions have been introduced.

4 Conclusions

The paper presented how the *J-CO-QL*$^+$ language (the query language of the *J-CO* Framework) is now capable to support "soft spatial queries": evaluation of membership degree to fuzzy sets is now integrated with soft spatial relations within the redesigned `JOIN` statement; it is now able to deal with membership degrees already available in joined documents; membership degrees to new fuzzy sets based on spatial relations now can be added too. The *J-CO-QL*$^+$ script presented in the paper addressed a problem of soft spatial querying in which authoritative data sets coming from various sources were integrated.

As a future work, the main activity will be devoted to complete the redesign of all the statements, so as to fully support soft querying. Specifically, we will address the concept of *fuzzy aggregator*, so as to define complex aggregations of

membership degrees in aggregated documents. Then, we plan to address multiple models of fuzzy sets, so as to deal with them in an integrated and unified way. The *J-CO* Framework is available on a public GitHub repository[4].

References

1. Castelltort, A., Laurent, A.: Towards fuzzy querying of NoSQL document-oriented databases. In: Laurent, A., Strauss, O., Bouchon-Meunier, B., Yager, R.R. (eds.) IPMU 2014. CCIS, vol. 444, pp. 384–395. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-08852-5_40
2. Blair, D.C.: Information retrieval, 2nd edn. c.j. van rijsbergen. london: Butterworths; 1979: 208 pp. price: $32.50. J. Am. Soc. Inf. Sci. **30**(6), 374–375 (1979). https://doi.org/10.1002/asi.4630300621
3. Bordogna, G., Ciriello, D.E., Psaila, G.: A flexible framework to cross-analyze heterogeneous multi-source geo-referenced information: The J-CO-QL proposal and its implementation. In: Proceedings of the International Conference on Web Intelligence, pp. 499–508 (2017)
4. Bordogna, G., Psaila, G.: Modeling soft conditions with unequal importance in fuzzy databases based on the vector p-norm. In: IPMU Conference, Malaga (2008)
5. Bordogna, G., Psaila, G.: Soft aggregation in flexible databases querying based on the vector p-norm. Int. J. Uncert. Fuzzi. Knowl. Based Syst. **17**(supp01), 25–40 (2009)
6. Bordogna, G., Psaila, G.: Customizable flexible querying in classical relational databases. In: Galindo, J. (ed.) Handbook of Research on Fuzzy Information Processing in Databases, pp. 191–217. IGI Global (2008)
7. Bosc, P., Prade, H.: An introduction to the fuzzy set and possibility theory-based treatment of flexible queries and uncertain or imprecise databases. In: Uncertainty Management in Information Systems, pp. 285–324. Springer, New York (1997). https://doi.org/10.1007/978-1-4615-6245-0
8. Bosc, P., Pivert, O.: SQLF: a relational database language for fuzzy querying. IEEE trans. Fuzzy Syst. **3**(1), 1–17 (1995)
9. Bosc, P., Pivert, O.: SQLF: query functionality on top of a regular relational database management system. In: Knowledge Management in Fuzzy Databases, pp. 171–190. Springer, Heidelberg (2000). https://doi.org/10.1007/978-3-7908-1865-9
10. Fosci, P., Marrara, S., Psaila, G.: Soft querying Geojson documents within the J-Co framework. In: 16th International Conference on Web Information Systems and Technologies (WEBIST 2020), pp. 253–265 (2020)
11. Fosci, P., Psaila, G.: Towards flexible retrieval, integration and analysis of JSON data sets through fuzzy sets: a case study. Information **12**(7), 258 (2021)
12. Galindo, J.: New characteristics in FSQL, a fuzzy SQL for fuzzy databases. WSEAS Trans. Inf. Sci. Appl. **2**(2), 161–169 (2005)
13. Galindo, J., Medina, J.M., Pons, O., Cubero, J.C.: A server for fuzzy SQL queries. In: Andreasen, T., Christiansen, H., Larsen, H.L. (eds.) FQAS 1998. LNCS, vol. 1495, pp. 164–174. Springer, Heidelberg (1998). https://doi.org/10.1007/BFb0055999
14. Galindo, J., Urrutia, A., Piattini, M.: Fuzzy Databases: Modeling, Design, and Implementation. IGI Global (2006)

[4] Github repository - https://github.com/JcoProjectTeam/JcoProjectPage.

15. Kacprzyk, J., Zadrożny, S.: Fquery for access: Fuzzy querying for a windows-based DBMS. In: Bosc, P., Kacprzyk, J. (eds.) Fuzziness in Database Management Systems. Studies in Fuzziness, vol. 5. Physica, Heidelberg (1995). https://doi.org/10.1007/978-3-7908-1897-0_18
16. Medina, J.M., Blanco, I.J., Pons, O.: A fuzzy database engine for mongoDB. Int. J. Intell. Syst. **37**, 5691–5724 (2022)
17. Medina, J.M., Pons, O., Vila, M.A.: Gefred: a generalized model of fuzzy relational databases. Inf. Sci. **76**(1), 87–109 (1994)
18. Psaila, G., Fosci, P.: Toward an analyist-oriented polystore framework for processing JSON geo-data. In: International Conferences on Applied Computing 2018, Budapest; Hungary, 21–23 October 2018. pp. 213–222. IADIS (2018)
19. Psaila, G., Fosci, P.: J-CO: a platform-independent framework for managing geo-referenced JSON data sets. Electronics **10**(5), 621 (2021)
20. Zadeh, L.A.: Fuzzy sets. Inf. Control **8**(3), 338–353 (1965)
21. Zadrozny, S., Kacprzyk, J.: Fquery for access: towards human consistent querying user interface. In: ACM Symposium on Applied Computing, pp. 532–536 (1996)